

Análise e Projeto de Software Orientado a Objetos - Planejamento de Projeto

Leonardo S. Victorio

Faculdade de Computação - UFMS

2026/1



- 1 Processo Unificado
- 2 O Sistema Livir
- 3 Planejamento na Fase de Concepção
- 4 Técnicas de Estimação de Esforço
- 5 Análise de Riscos
- 6 Pontos de Caso de Uso (UCP)
- 7 Planejamento Iterativo
- 8 Conclusão



O Processo Unificado (UP)

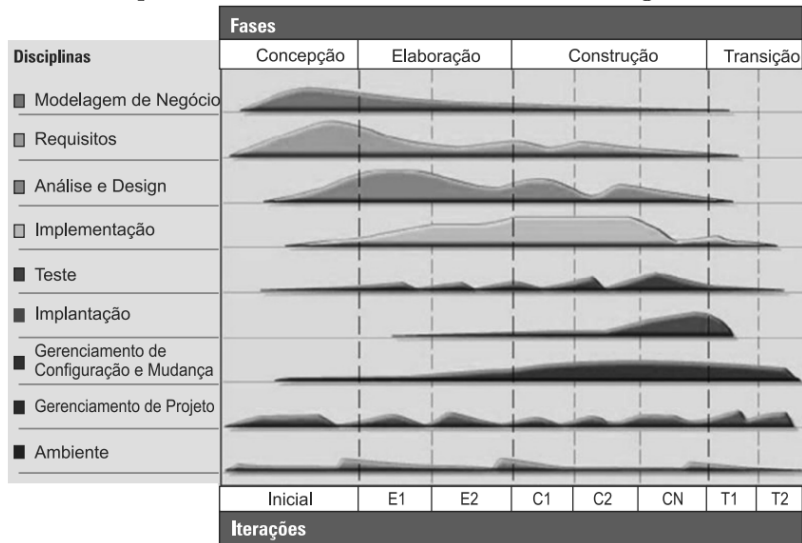
O **Processo Unificado** (UP) organiza o desenvolvimento de software em **quatro fases** e múltiplas **disciplinas** (fluxos de trabalho).

Fase	Objetivo principal
Concepção	Descobrir requisitos principais, compreender o escopo, avaliar viabilidade e elaborar o plano inicial.
Elaboração	Expandir os casos de uso prioritários, estabilizar a arquitetura e mitigar os maiores riscos.
Construção	Produzir o código da aplicação completa e implementar as mudanças solicitadas.
Transição	Realizar testes finais, entregar o sistema, instalar e migrar dados.



Disciplinas × Fases — O Gráfico Hump

Cada disciplina tem **intensidade variável** ao longo das fases.



Cada disciplina tem **intensidade variável** ao longo das fases.

Ideia central

O UP **não** é sequencial (Cascata): todas as disciplinas estão ativas em todas as fases, mas com **ênfases diferentes**.

Exemplos:

- **Requisitos** têm pico na Concepção e Elaboração.
- **Implementação** domina a Construção.
- **Gerenciamento de projeto** é constante em todas as fases.



Iterações no Processo Unificado

As fases de **Elaboração** e **Construção** são realizadas em **iterações**.

O que é uma iteração?

Um ciclo com início e fim bem definidos que produz uma **versão executável** do sistema (entrega interna ou ao cliente). Em cada iteração, a equipe:

- 1 Expande e implementa um ou mais **casos de uso**.
- 2 Mitiga **riscos** selecionados.
- 3 Incorpora **mudanças** solicitadas em iterações anteriores.

Diferença para o Waterfall

No UP, *cada iteração* percorre análise, design, implementação e teste. O produto cresce **incrementalmente** a cada ciclo.



Entregáveis da Fase de Concepção

Ao final da Concepção, a equipe deve ter produzido:

- **Modelo conceitual preliminar** — principais entidades do domínio.
- **Documento de requisitos** — lista de casos de uso de alto nível e especificações suplementares.
- **Cronograma de desenvolvimento** — baseado na análise de pontos de caso de uso.
- **Lista de riscos de alta exposição** e planos de mitigação.
- **Plano da primeira iteração** da Elaboração.

Foco desta aula

Aprender a elaborar o **cronograma** e os **planos de risco** — os itens que ainda não foram abordados nas aulas anteriores.



Visão do produto

Uma **livraria virtual**: o sistema gerencia os processos de **aquisição** e **venda** de livros. O acesso de clientes e gerentes se dá por *website*.

Funcionamento

- Livro disponível em estoque → entrega imediata.
- Fora de estoque → pedido à editora com prazo informado ao cliente.
- Sistema calcula impostos, frete e descontos.
- Gera relatórios de vendas e sugestões ao cliente.

Restrições (v1.0)

- Pagamento apenas por **cartão de crédito**.
- Apenas **livros novos** (sem usados).
- Acesso somente via **website**.

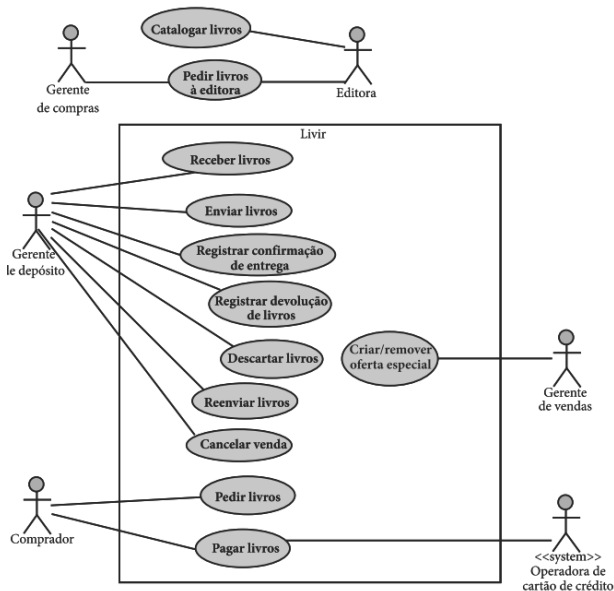


Ator	Tipo
Comprador	Humano
Gerente de vendas	Humano
Gerente de compras	Humano
Gerente de depósito	Humano
Operadora de cartão « <i>system</i> »	Sistema (protocolo)

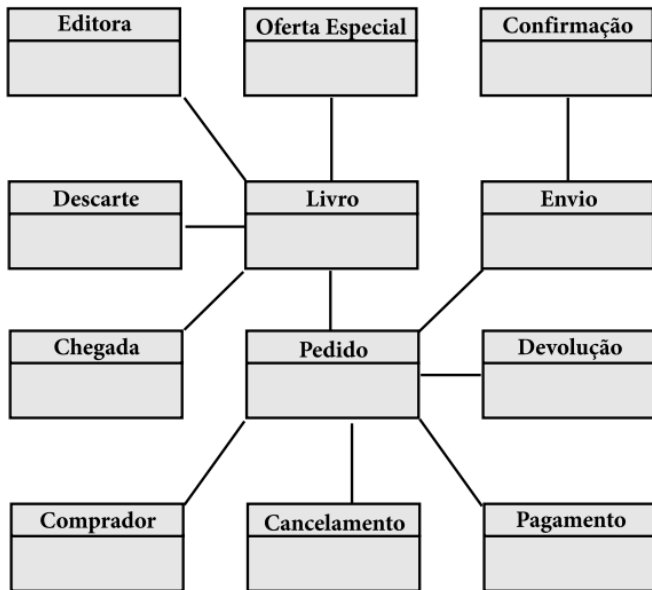
Observação

A operadora de cartão de crédito é um sistema externo que interage via **protocolo de rede**.

Livir — Diagrama de Casos de Uso



Livir — Modelo Conceitual Preliminar



Onde Estamos no Processo Unificado

Nas aulas anteriores levantamos os casos de uso do sistema.

Agora, ainda na **fase de Concepção**, precisamos responder:

- **Quanto** tempo e esforço o projeto vai exigir?
- **Quais** são os principais riscos?
- **Como** organizar o desenvolvimento em iterações?

Atividade central desta aula

Planejamento de projeto baseado em casos de uso: estimação de esforço, análise de riscos e definição das iterações.

Por que planejar aqui?

É na Concepção que se decide se o projeto **vale a pena** seguir adiante. Um plano realista é o principal entregável desta fase.

Por que Projetos de Software Falham?

Historicamente, a maioria dos projetos de software:

Problemas comuns

- Demoram **mais** do que o previsto.
- Custam **mais** do que o orçado.
- Entregam **menos** funcionalidades.
- Têm **qualidade** inferior ao esperado.

Objetivo da estimaco

Fazer com que equipe e clientes aprendam a **esperar** o tempo, custo, qualidade e escopo **corretos** para o seu projeto e sua equipe.

Consequncia

Tcnicas de estimaco de esforo existem para tornar as expectativas **realistas** antes que compromissos sejam assumidos.

Plano de Fase vs. Plano de Iteração

Durante a Concepção constroem-se dois tipos de plano:

Plano de fase Visão geral do projeto inteiro: fases, marcos principais, esforço total, prazo e tamanho da equipe.

Plano de iteração Detalha **uma** iteração de cada vez: quais casos de uso serão implementados, quais riscos serão mitigados e quais tarefas cada membro da equipe realizará.

Regra do Processo Unificado

Na Concepção, elabora-se o plano de fase **e** o plano da **primeira iteração** da Elaboração. Cada iteração seguinte é planejada enquanto a anterior está em andamento.



Duas Grandes Classes de Técnicas

Técnicas *não paramétricas*

Não se baseiam em medidas do projeto. Dependem de experiência, intuição ou negociação.

- Opinião de especialista
- Estimação por analogia
- Lei de Parkinson
- Preço para vencer
- Pontos de história (ágil)

Técnicas *paramétricas*

Baseiam-se em medidas concretas do projeto: tamanho, complexidade e capacidade da equipe.

- COCOMO II (linhas de código)
- Análise de Pontos de Função
- **Pontos de Caso de Uso**

Princípio

Quanto mais **dados históricos** e **padrões consistentes** uma equipe possui, melhores serão as estimativas — independentemente do método.

Técnicas Não Paramétricas — Pontos Positivos e Negativos

Opinião de especialista Rápida e barata se os especialistas têm experiência em projetos similares; pouco precisa caso contrário.

Estimação por analogia Base pragmática da opinião de especialista: compara o novo projeto com projetos passados semelhantes. Inviável sem histórico de projetos.

Lei de Parkinson *“O projeto vai custar a quantidade de recursos disponível.”* Vantagem: nunca estoura o orçamento. Desvantagem: frequentemente o escopo fica aquém do esperado.

Preço para vencer Custo igual ao que se acredita ganhar o contrato. Pode comprometer qualidade e escopo.



Definição

Um **ponto de história** representa um *dia ideal de trabalho* (6–8 horas focadas). A equipe estima cada história de usuário em pontos.

A atribuição considera três dimensões:

- **Complexidade:** quantos cenários possíveis a história possui?
- **Esforço:** a mudança afeta muitos componentes diferentes?
- **Risco:** alguém na equipe domina a tecnologia necessária?

Escala de Fibonacci aproximada

0, $\frac{1}{2}$, 1, 2, 3, 5, 8, 13, 20, 40, 100...

A escala não linear reflete que estimativas ficam menos precisas para histórias maiores. Histórias acima de ~ 10 pontos devem ser **divididas**.

Técnica	Base de medição	Quando usar
COCOMO II	Linhas de código (KSLOC)	Projetos com histórico de KSLOC disponível
Pontos de Função (FPA)	Requisitos funcionais, classes e transações	Quando requisitos estão bem definidos
Pontos de Caso de Uso (UCP)	Casos de uso: atores e complexidade	Processos iterativos baseados em UC

Limitação do COCOMO II

Exige estimar KSLOC *antes* de escrever o código — grande fonte de incerteza. Pontos de Função e UCP partem dos **requisitos**, conhecidos desde o início.

O que é um Risco?

Definição

Um **risco** é algo que *pode* acontecer e causar problemas no projeto. É composto por três partes:

Causa Condição incerta que pode criar problemas.

Problema Situação que ocorreria dada a incerteza da causa.

Efeito Impacto sobre um ou mais objetivos do projeto.

Exemplo

Causa: equipe sem experiência em pagamentos on-line.

Problema: integração com operadora de cartão mal implementada.

Efeito: erros em transações financeiras, atraso na entrega.



- 1 **Identificação** — descobrir as condições que podem iniciar problemas. *Checklists* estruturados (como o de Carr et al., 1993) ajudam.
- 2 **Análise** — determinar **probabilidade** e **impacto** de cada risco.
- 3 **Planejamento** — criar **planos de mitigação** e **contingência** para riscos de alta e média exposição.
- 4 **Controle** — monitorar riscos ao longo *de todo* o projeto; riscos mudam mais rapidamente do que requisitos.

Atenção

Riscos são **mais instáveis** do que requisitos. Novos riscos surgem durante o desenvolvimento e a exposição dos existentes pode mudar.



Categorias de Risco (Carr et al., 1993)

Engenharia do produto

- Requisitos instáveis
- Design difícil
- Implementação inviável
- Integração inadequada

Ambiente de desenvolvimento

- Processo mal definido
- Ferramentas inadequadas
- Equipe sem treinamento
- Gerência inexperiente

Restrições externas

- Cronograma irreal
- Orçamento insuficiente
- Contratos restritivos
- Comunicação fraca com o cliente

Observação

A lista é apenas uma **referência inicial** — riscos imprevisíveis específicos ao domínio são identificados ao longo do projeto.

Avaliação de Probabilidade e Impacto

Utiliza-se a **escala camiseta** (grande / médio / pequeno):

Probabilidade

- **Grande** — quase certo que ocorra.
- **Médio** — pode ocorrer eventualmente.
- **Pequeno** — chance mínima.

Impacto

- **Grande** — pode cancelar o projeto.
- **Médio** — aumenta significativamente os custos.
- **Pequeno** — transtorno, perdas recuperáveis.

Impacto \ Prob.	Grande	Médio	Pequeno
Grande	Alta	Alta	Média
Médio	Alta	Média	Baixa
Pequeno	Média	Baixa	Baixa



Planos de Mitigação e Contingência

Para cada risco de **alta** ou **média** exposição elaboram-se planos:

Mitigação de probabilidade Atua na *causa* — reduz a chance de o risco ocorrer. *Ex.: contratar arquiteto experiente para mitigar inexperiência da equipe.*

Mitigação de impacto Atua nos *efeitos* — reduz o prejuízo caso o risco ocorra. *Ex.: modularização e controle de versão para requisitos instáveis.*

Contingência Atua no *problema já ocorrido* — o que fazer depois. *Ex.: sistema de gerenciamento de mudanças para controlar alterações de requisitos.*

Importante

Planos de mitigação são **preventivos** (antes do problema). Planos de contingência são **reativos** (após o problema). Riscos de **baixa** exposição devem apenas ser **monitorados**.

A técnica de **Pontos de Caso de Uso** (Karner, 1993) estima o esforço a partir da quantidade e complexidade dos atores e casos de uso.

Fluxo de cálculo

$$\underbrace{\text{UAW} + \text{UUCW}}_{\text{UUCP}} \xrightarrow{\times \text{TCF}} \xrightarrow{\times \text{EF}} \text{UCP} \xrightarrow{\times \text{TPI}} \text{Esforço (E)}$$

UAW	Peso dos atores não ajustado
UUCW	Peso dos casos de uso não ajustado
TCF	Fator de complexidade técnica
EF	Fator ambiental
TPI	Índice de produtividade da equipe

UAW — Peso dos Atores Não Ajustado

Cada ator é contado **uma única vez**. Apenas as especializações mais elementares são contadas (sem subtipos).

Tipo de ator	Pontos
Humano com interface gráfica	3
Sistema ou humano via linha de comando	2
Sistema via API	1

Exemplo — Projeto Livir

Gerente do depósito (3) + Gerente de compras (3) + Gerente de vendas (3) + Comprador (3) + Operadora de cartão «*system*» (2)

$$UAW = 3 + 3 + 3 + 3 + 2 = 14$$

Observação

O uso do UAW é **opcional**: algumas equipes optam por excluí-lo, fazendo $UUCP = UUCW$.

UUCW — Peso dos Casos de Uso Não Ajustado

Apenas **processos completos** são contados (extensões e fragmentos, não).

Complexidade	Critério (risco/tipo)	Pontos
Simples	Relatórios — baixo risco, apenas leitura	5
Médio	CRUDs — risco médio, lógica conhecida	10
Complexo	Não padronizado — alto risco, lógica desconhecida	15

Exemplo — Projeto Livir

8 casos de uso *report* $\rightarrow 8 \times 5 = 40$

3 casos de uso CRUD $\rightarrow 3 \times 10 = 30$

10 casos de uso sem estereótipo $\rightarrow 10 \times 15 = 150$

$$UUCW = 40 + 30 + 150 = 220$$

UUCP — Pontos de Caso de Uso Não Ajustados

$$UUCP = UAW + UUCW$$

No exemplo Livir: $UUCP = 14 + 220 = 234$

TCF — Fator de Complexidade Técnica

13 fatores técnicos (T1–T13), cada um com nota 0–5 e peso próprio.

$$TFactor = \sum_{i=1}^{13} peso_i \times nota_i \quad (0 \leq TFactor \leq 70)$$

$$TCF = 0,6 + 0,01 \times TFactor$$

$TCF < 1$ indica projeto tecnicamente **mais simples** que o nominal;

$TCF > 1$ indica projeto tecnicamente **mais complexo**.

Exemplo Livir

$TFactor = 22,5 \Rightarrow TCF = 0,6 + 0,01 \times 22,5 = 0,825$ (82,5% do nominal)



Fatores Técnicos (T1–T13)

Cód.	Fator	Peso
T1	Sistema distribuído	2
T2	Tempo de resposta / desempenho	1
T3	Eficiência do usuário final	1
T4	Complexidade de processamento interno	1
T5	Design visando reusabilidade de código	1
T6	Facilidade de instalação	0,5
T7	Facilidade de operação	0,5
T8	Portabilidade	2
T9	Design visando facilidade de manutenção	1
T10	Processamento concorrente/paralelo	1
T11	Segurança	1
T12	Acesso de/para código de terceiros	1
T13	Necessidades de treinamento de usuários	1



Diferencial do UCP

O mesmo projeto pode ter UCPs diferentes para equipes diferentes. O **EF** captura a qualidade do ambiente de desenvolvimento.

Cód.	Fator	Peso
E1	Familiaridade com o processo de desenvolvimento	1,5
E2	Experiência na aplicação	0,5
E3	Experiência com orientação a objetos	1
E4	Experiência do analista líder	0,5
E5	Motivação	1
E6	Estabilidade de requisitos	2
E7	Trabalhadores em tempo parcial (<i>negativo</i>)	-1
E8	Dificuldade com a linguagem de programação (<i>neg.</i>)	-1

$$EFactor = \sum_{i=1}^8 peso_i \times nota_i \quad EF = 1,4 - 0,03 \times EFactor$$



UCP, Esforço, Tempo Linear e Tamanho da Equipe

UCP — Pontos de Caso de Uso Ajustados

$$UCP = UUCP \times TCF \times EF$$

Exemplo Livir: $UCP = 234 \times 0,825 \times 1,355 \approx 261,6$

Esforço total

$$E = UCP \times TPI \quad (\text{funcionário-mês})$$

TPI típico: 0,095 a 0,228 funcionário-mês/UCP.

Karner (1993) sugeriu 0,127 como ponto de partida sem histórico.

Tempo linear e tamanho médio da equipe

$$T = 2,5 \times \sqrt[3]{E} \quad P = \frac{E}{T}$$

Exemplo Livir: $E \approx 59,6 \Rightarrow T \approx 9,8$ meses, $P \approx 6$ pessoas

Determinando o TPI pela Regra de Schneider & Winters

Conte os **fatores ambientais ruins**:

- E1–E6 com nota **abaixo de 3** (fatores positivos insatisfatórios).
- E7–E8 com nota **acima de 3** (fatores negativos agravados).

Total de fatores ruins	TPI recomendado
≤ 2	20 h/UCP (0,127 func.-mês/UCP)
3 ou 4	28 h/UCP (0,177 func.-mês/UCP)
> 4	36 h/UCP (0,228 func.-mês/UCP)
	<i>Recomenda-se melhorar o ambiente antes de assumir qualquer compromisso.</i>

Exemplo Livir — equipe de iniciantes

5 fatores ruins (E1, E2, E4, E6, E8) $\Rightarrow$ TPI = 0,228

$E = 261,6 \times 0,228 \approx 59,6$ funcionários-mês

Duração das Iterações

Uma iteração começa com planejamento e termina com uma **entrega interna ou ao cliente**.

Tamanho da equipe	Duração recomendada
Até ~5 pessoas	1 semana
Até ~20 pessoas	3–4 semanas
Acima de 40 pessoas	6–8 semanas

Outros fatores que reduzem a duração ideal:

- Geração automática de código e ferramentas integradas.
- Equipe experiente em análise, design e codificação.

Fatores que aumentam a duração:

- Testes e revisões extensos (qualidade crítica).



Número de Iterações e Distribuição das Fases

Número de iterações

$$n_{iter} = \frac{T}{\text{duração da iteração}}$$

Exemplo Livir (10 meses, iterações de 3 semanas $\approx 0,75$ mês):
 $n_{iter} \approx 13,3 \rightarrow 14$ iterações

Distribuição típica das fases (Kruchten, 2003):

Fase	% do tempo	Exemplo (10 meses)
Concepção	10%	~ 4 semanas
Elaboração	30%	~ 4 iterações
Construção	50%	~ 7 iterações
Transição	10%	~ 4 semanas



Capacidade de Carga da Equipe (TLC)

Definição

$$TLC = P \times \text{duração da iteração (meses)}$$

TLC representa o total de **funcionários-mês disponíveis** em uma iteração.

Regra de uso

O esforço total atribuído a uma iteração **não pode superar** a TLC. Se um caso de uso complexo exige mais do que a TLC, duas opções existem:

- 1 Aumentar a duração da iteração.
- 2 Dividir o caso de uso em partes menores.



Capacidade de Carga da Equipe (TLC)

Definição

$$TLC = P \times \text{duração da iteração (meses)}$$

TLC representa o total de **funcionários-mês disponíveis** em uma iteração.

Exemplo

Equipe de 6 pessoas, iterações de 3 semanas ($\approx 0,75$ mês):

$$TLC = 6 \times 0,75 = 4,5 \text{ funcionários-mês por iteração}$$



Esforço por Tipo de Caso de Uso

Para alocar casos de uso a iterações, calcula-se o esforço por ponto:

$$E_{UCP} = \frac{E}{UUCW} \quad (\text{funcionário-mês por ponto})$$

Depois multiplica-se pelo peso do tipo de caso de uso:

Tipo	Pontos	Esforço (equipe Livir)
Simples	5	$5 \times 0,271 = 1,36$ func.-mês
Médio	10	$10 \times 0,271 = 2,71$ func.-mês
Complexo	15	$15 \times 0,271 = 4,07$ func.-mês

Como usar

Compare o esforço de cada caso de uso com a TLC da iteração para saber quantos e quais casos de uso cabem em cada iteração.

Priorização dos Casos de Uso

A ordem de implementação segue três critérios:

Complexidade e risco Casos de uso **complexos e arriscados primeiro** — a equipe aprende mais e a arquitetura estabiliza mais cedo.

Dependências CUs fortemente dependentes entre si devem ser tratados **na mesma iteração**.

Valor de negócio Processos **críticos ao negócio** têm prioridade.

Consequência

CRUDs e relatórios ficam para a **Construção**, após a arquitetura estar estável.



Objetivos de uma Iteração

Os objetivos de cada iteração podem ser de três tipos:

- 1 **Implementar** total ou parcialmente um ou mais casos de uso.
- 2 **Mitigar** um ou mais riscos (plano de redução de probabilidade, de impacto ou de contingência).
- 3 **Incorporar mudanças** solicitadas — correções de não conformidades ou erros de design descobertos em iterações anteriores.

Mitigação e *time box*

Atividades de mitigação de risco nem sempre são estimáveis em pontos de caso de uso. Nesse caso, atribui-se uma **caixa de tempo** (*time box*) fixa. Ao final, avalia-se se a atividade deve continuar.



Plano de Fase — Estrutura Geral

Fase	Principais objetivos	Crono.
Concepção	Visão, escopo, arquitetura candidata, casos de uso de negócio, plano do projeto	Sem. 1–4
Elaboração E1	Instalar arquitetura, validar requisitos, implementar CU prioritários	Sem. 5–7
Elaboração E2–E4	Mitigar riscos arquiteturais, testar carga, completar arquitetura	Sem. 8–16
Construção C1–C7	Implementar CU restantes, integrar e validar, planejar beta	Sem. 17–37
Transição T1	Versão beta, correções finais, manual do usuário, entrega ao cliente	Sem. 38–40



- **Planejamento** é uma atividade central da Concepção no Processo Unificado.
- Técnicas **não paramétricas** (opinião, analogia, pontos de história) são rápidas, mas dependem fortemente de experiência.
- Técnicas **paramétricas** (UCP, FPA, COCOMO II) partem de medidas objetivas do projeto e produzem estimativas mais reproduzíveis.
- **UCP** é natural para o PU: baseia-se diretamente nos casos de uso já levantados na Concepção.
- **Riscos** devem ser identificados, analisados e gerenciados ao longo de *todo* o projeto.
- O **plano de fase** e o **plano da primeira iteração** são os principais entregáveis de planejamento da Concepção.



Resumo do Fluxo de Estimação com UCP

- 1 Identificar e classificar **atores** (UAW).
- 2 Classificar **casos de uso** por complexidade (UUCW).
- 3 Calcular $UUCP = UAW + UUCW$.
- 4 Avaliar os **fatores técnicos** (T1–T13) $\rightarrow$ TCF.
- 5 Avaliar os **fatores ambientais** (E1–E8) $\rightarrow$ EF.
- 6 Calcular $UCP = UUCP \times TCF \times EF$.
- 7 Determinar o **TPI** (histórico ou regra de Schneider & Winters).
- 8 Calcular $E = UCP \times TPI$, $T = 2,5\sqrt[3]{E}$, $P = E/T$.
- 9 Definir duração de iteração $\rightarrow$ número de iterações $\rightarrow$ TLC.
- 10 Priorizar casos de uso e distribuir nas iterações.



O que Vem a Seguir

Com o plano de projeto em mãos, a equipe inicia a **Elaboração**:

- **Expansão dos casos de uso** prioritários — detalhamento dos fluxos principal e alternativos.
- **Diagramas de sequência** — como os objetos colaboram para realizar cada caso de uso.
- **Diagramas de classes de design** — evolução do modelo conceitual com atributos, métodos e visibilidade.
- **Estabilização da arquitetura** — definição dos componentes e suas interfaces.

Lembre-se

O planejamento é **vivo**: revise estimativas e prioridades a cada iteração. O índice de produtividade (TPI) deve ser **calibrado** com os dados reais do projeto em andamento.

Referências Bibliográficas

-  WAZLAWICK, Raul Sidnei. *Análise e Projeto de Sistemas de Informação Orientados a Objetos: modelagem com UML, OCL e IFML*. 3. ed. Rio de Janeiro: Elsevier, 2015. Cap. 4 — Planejamento de projeto baseado em casos de uso.
-  KARNER, Gustav. *Resource Estimation for Objectory Projects*. Objective Systems SF AB, 1993.
-  SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson Education do Brasil, 2019. Cap. 22 — Gerenciamento de projetos.
-  BOEHM, Barry W. *Software Cost Estimation with COCOMO II*. Upper Saddle River: Prentice Hall, 2000.

Dúvidas e Discussões

Leonardo S. Victorio

leonardo.victorio@ufms.br

Faculdade de Computação — UFMS

“Uma equipe bem afinada, com registros históricos e padrões consistentes, produzirá estimativas melhores do que uma equipe inexperiente, não importa qual método use.”

— Wazlawick (2015)

